Mission 2a

Now that the telemetry is working, you check the photos the drone has taken of the planet (Jimbo ordered a nice wallpaper before he left, and you consider it the most important part of the mission). Unfortunately, *the colors are completely wrong!* The camera's lenses are uncalibrated.

Thankfully, you can use the pre-loaded *hyper-photon calibration utility* to calibrate the camera yourself. It flashes a fake photo into the lens, and at the end the colors should be completely neutral. The first step is to find the camera's **divergence** from the expected result.

Before you get started, you take a look at the camera's manual. Three things stand out:

- The camera is a row of lenses. Each lens has a state from **`−3`** (very cold/blue) to **`+3`** (very hot/red), and they all start at **`0`** (neutral).
- Photons may be **hot, cold or neutral** depending on their wavelength. In each frame, a hot photon `h` heats a lens by 1, a cold photon `c` cools it by 1, and `-` does nothing. However, **heating a very hot lens (+3) loops it round to being very cold (−3)**. In the same way, going past −3 loops round to +3.
- The camera was built using *fine-grained quantum glass!* Each lens may be **entangled** with some of its neighbours, meaning that a photon landing on a lens also **hits every lens up to its *entanglement value* away.** A photon on lens 5 with *entanglement value* 2 hits lenses 3 to 7. The lenses it reaches don't spread it any further, and no photon ever reaches past either end of the row.

You already have the quantum-entanglement values, and the *calibration utility* gives you the indicative photons that would hit your camera frame by frame ***(both are your puzzle input)***. Below is an example:

```
0 1 0 0 1 0
--h---
-hhc-c
--hcc-
h-chhc
```

The lenses after each frame are:

```
start    0  0  0  0  0  0
frame 1  0  0  1  0  0  0
frame 2  1  1  3 -1  0 -1
frame 3  1  1 -3 -3 -1 -2
frame 4  2  1  3 -1  0 -2
```

- **Frame 1**: the `h` lands on *lens 3*, which has *entanglement value* `0`, so only *lens 3* changes.
- **Frame 2**: the `h` on *lens 2* (*entanglement value 1*) heats *lenses 1 to 3*, and the `h` on *lens 3* heats *lens 3* again, taking it to +3. The `c`s on *lenses 4 and 6* cool only those lenses.
- **Frame 3**: the `h` on *lens 3* loops it from +3 round to −3. The `c` on *lens 5* (*entanglement value 1*) cools *lenses 4 to 6*, so *lens 4* is cooled twice, once by its own `c`.
- **Frame 4**: the `h` on *lens 1* heats only *lens 1*. The `c` on *lens 3* loops it the other way, from −3 round to +3. The `h` on *lens 5* (*entanglement value 1*) heats *lenses 4 to 6*, so *lens 4* is heated twice, and on *lens 6* it cancels out the `c`.

Here is a larger example:

```
0 0 2 2 2 0 1 0 0
-------h-
--cc---c-
------h--
c---c-h-h
```

```
---c--h-c
```

The lenses after each frame are:

```
lens     1  2  3  4  5  6  7  8  9
start    0  0  0  0  0  0  0  0  0
frame 1  0  0  0  0  0  0  0  1  0
frame 2 -1 -2 -2 -2 -2 -1  0  0  0
frame 3 -1 -2 -2 -2 -2  0  1  1  0
frame 4 -2 -2 -3 -3 -3  0  1  2  1
frame 5 -2 -3  3  3  3  0  2  3  0
```

- **Frame 1**: the `h` on *lens 8* (*entanglement value 0*) only heats *lens 8*.
- **Frame 2**: the `c` on *lens 3* (*entanglement value 2*) cools *lenses 1 to 5*, and the `c` on *lens 4* (*entanglement value 2*) cools *lenses 2 to 6*. *Lenses 2 to 5* are hit by both. The `c` on *lens 8* cancels frame 1's heat.
- **Frame 5**: *lenses 3, 4 and 5* are already at −3 when the `c` on *lens 4* cools them again, so they loop round to +3.

On the calibration photo, the camera should end neutral. To measure how far off it is, you use its **divergence number**: multiply each lens's final state by its position (counted from 1), and add it all up.
In the first example, the **divergence number** is `1×2 + 2×1 + 3×3 + 4×(−1) + 5×0 + 6×(−2) = −3` *(it can be negative)*. In the larger example, it is `1×(−2) + 2×(−3) + 3×3 + 4×3 + 5×3 + 6×0 + 7×2 + 8×3 + 9×0 = 66`. ***What is the divergence number for your camera?***